

GRAFT

**Grounded Repair of Attack-graph
Traversal** — recovering from
capability loss in autonomous
penetration testing, and admitting
success only on captured evidence.

AUTHOR

Austin

ADVISOR

Prof. Didik

TARGET VENUE

IEEE ICC 2027

SYSTEM

lgg_automation · graph-orchestrator

An autonomous agent's step can fail two ways — and agents treat them the same.

The whole recovery strategy depends on telling them apart. Conflating them is why LLM pentest agents get stuck or lie about it.

technique failure

The action was wrong

Wrong exploit module, wrong option. The world state is intact — so **retry** a different technique and move on. Ordinary replanning handles this fine.

capability loss

The action was fine — a precondition vanished

The **session died** (blip, reboot, kill). Every downstream step that needed it is *orphaned*. Retrying is useless — the session is dead at *every* retry budget.

“Capability orphaning” (Eq. 6): when a held capability is lost, its precondition is unsatisfiable at any retry budget — so repair must be **structural**, not a retry.

Detect the difference, repair it structurally, and never trust the agent's word.

C1 · the graft

Structural repair of capability loss

A feasibility gate detects that a *held* capability is gone, re-provisions it (grows a fresh re-exploit), and **re-parents** the orphaned objective onto it. A graph mutation – not a retry.

C2 · the grounding oracle

Success = captured target bytes

uid=0(root) in captured output; a proof marker *read back off disk*.
Never the agent's prose. Formal property $\Delta_A = \emptyset$: no unproven claim ever enters execution state.

The system beneath them – a backtracking walker over an attack graph, driving Metasploit + SSH against a lab target – is the vehicle. These two ideas are the science.

A loop of three parts: a history **H**, a gate **F**, and a structural repair.

- **H – capability history.** After every success, record what was established (session, root, artifact). Monotonic: “ever held”, from concrete fields, never prose.
- **F – feasibility gate.** Before each node, is the required capability live? Missing & in H → **capability** loss. Missing & not in H → **precondition** (mis-order).
- **The repair.** On capability loss: grow a fresh re-exploit, re-parent the orphaned node(s) onto it, reset to pending. The objective survives.

```

✓ recon → services mapped
|
✓ gain_access → session 1 · H += {session}
|
✗ harness kills session 1
|
F file_drop → F=0 · req {session} world {} · ∈ H →
  capability
|
  graft grow re_exploit · re-parent file_drop
|
✓ re_exploit → session 2 · F passes
|
✓ file_drop on session 2 → marker read back → grounded

```

F fires exactly once; one graft; no loop. Turn the graft off and the re-exploit is never grown – the session stays dead and the objective is silently abandoned.

core_agents/orchestrator.py

Success is defined over the raw bytes the target emitted.

An LLM will happily report “I am now root” when it isn't. If that self-report drives execution state, the whole run is built on a lie.

rejected the agent's word

A plan line that merely *mentions* `uid=0(root)`. A stage's self-set `new_level`. Circular – defeats the metric.

admitted captured evidence

A root token co-occurring with a real probe line (`DIRECT ID CHECK`). The exact marker written, then *read back* in a `cat` output.

The oracle is re-derived from run artifacts by `experiments/parse_eval.py` – independent of the acting agent. The money metric **false_success** counts a claimed success with no captured proof.

A backtracking walker over an attack graph, with three layers of decision.

recon → initial_access → privesc → persistence → impact

L1

tactical

Stage subagents (stages/*.py)

Each a Planner→Executor→Critic loop. Tight kill-chain scope, broad tools inside it.

L2

strategic

The judge — judge()

Between nodes: continue · adapt · escalate. Catches drift early.

L3

heavyweight

The replanner — _replan_from()

Graph mutation. LLM emits a tiny intent; code expands it. **The graft lives here.**

The LLM enters at L2/L3 — never inside the deterministic edge routing. That's what keeps every run replayable and auditable.

Two scenarios, two variants, a minimal-diff ablation.

Scenarios

`orphan_c` the headline – kill the session right before the impact objective. Recovery is possible only via the graft.

`flaw_privesc` the control – a wrong exploit module at `privesc`. A *technique* failure that substitution alone repairs.

Variants

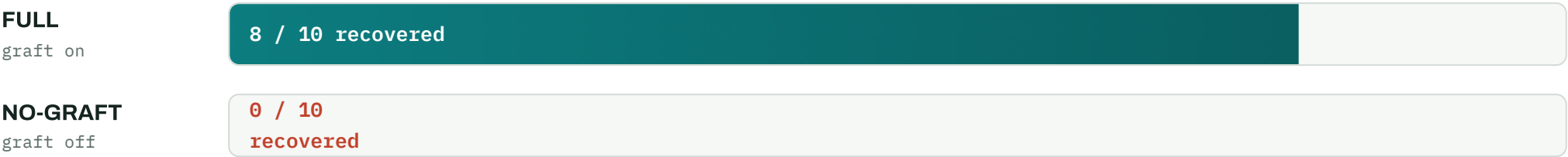
`v0_full` everything on.

`v_nograft` identical but the capability branch routes to technique-substitution instead of the graft – **one flag** (`EVAL_ENABLE_GRAFT=0`).

Lab: Metasploitable 3 target (192.168.34.7), Kali + msfrpcd, GPT-4o judge/replanner. Fault injected by a `pre_node_hook` that kills the session before the orphaned node. **N = 10** per cell.

On capability loss, the graft is the difference between recovery and none.

RESULT 1 · RECOVERY FROM CAPABILITY LOSS — orphan_c



Fisher exact (2-sided) p = 0.0007

Not “lower” — a clean zero. Without the graft the lost session cannot be restored, so recovery is impossible at any budget.

RESULT 2 · CONTROL — flaw_privesc

VARIANT	GROUNDED
FULL	7 / 8
NO-GRAFT	8 / 10

No separation. The 8-vs-0 gap appears *only* on capability loss — so it is the graft, not generic replanning.

RESULT 3 · GROUNDING

0 / 38

FALSE SUCCESSES
ACROSS ALL VALID RUNS

$\Delta_A=0$

VERIFIED 3 WAYS:
EMPIRICAL · ADVERSARIAL ·
FORMAL

What's proven, and what is deliberately future work.

- **C1 – the graft: complete & significant.** 8/10 vs 0/10, $p = 0.0007$, with a matched control.

- **C2 – the oracle's soundness half: proven.** $\Delta_A = 0$ empirically (0/38), by an offline adversarial test, and by construction.

- **C2's complementary spike ($\Delta_A = \Delta_R$): not yet shown.** Our own stages rarely overclaim (privesc honestly times out). The right vehicle is an *external* agent that does – future work.

- **Generality across placements: one clean placement so far.** orphan-B (a second placement) is scaffolded and ready to run.

Threats to validity, controlled: lab artifacts excluded by protocol (not concentrated in the claim variant); grounding independent of the acting agent; single lab / single target model is the standing limitation.

Where to pick it up — in priority order.

next · scaffolded

orphan-B — generality

Kill the session before *persistence*: a second placement, with a terminal impact node so the objective still grounds independently. Stronger than orphan-C — the graft must re-parent a **two-node subgraph**. Graph + offline test already written.

next · the C2 spike

External agent — HackingBuddyGPT

Run a third-party agent that genuinely overclaims through *our* oracle, unmodified. Cheap first step: 3 transcripts — does it claim root when captured id doesn't show it? If yes, the spike is real.

Start here → [docs/ONBOARDING.md](#) then HANDOFF.md · DESIGN_PHILOSOPHY.md results → [reports/graft_results_summary.md](#)

read side by side → orphan_c_graph.py + parse_eval.py